

netcat(1) Name nc, netcat - arbitrary TCP and UDP connections and listens

Synopsis nc -h

nc [-46dnrtuvz] [-i interval] [-P proxy_username] [-p port] [-s source_ip_address] [-T dspc] [-w timeout] [-X proxy_protocol] [-x proxy_address[:port]] [-L timeout] [-e program] [-b bufsize] [-q timeout] [-m bytes] [-l bufsize] [-O bufsize] [-S sla-prop] hostname port_list nc -l [-46DdEFnrtuvzZ] [-i interval] [-T dspc] [-e program] [-b bufsize] [-q timeout] [-R address/port[/proto]] [-m bytes] [-L timeout] [-l bufsize] [-O bufsize] [-S sla-prop] [hostname] port nc -l [-46DEFdnrtuvzZ] [-i interval] [-T dspc] [-e program] [-b bufsize] [-q timeout] [-R address/port[/proto]] [-m bytes] [-L timeout] [-l bufsize] [-O bufsize] [-S sla-prop] -p port [hostname] nc -U [-Ddtvz] [-i

interval] [-w timeout] [-e program] [-b
bufsize] [-q timeout] [-m bytes] path nc -Ul
[-46DdktvZ] [-i interval] [-e program] [-b
bufsize] [-q timeout] [-R
address/port[/proto]] [-m bytes] path

Description The nc (or netcat) utility is used for a variety of tasks associated with TCP or UDP. nc can open TCP connections, send UDP packets, listen on arbitrary TCP and UDP ports, perform port scanning, and deal with both IPv4 and IPv6. Unlike telnet(1), nc scripts nicely, and separates error messages onto standard error instead of sending them to standard output.

The nc command is often used for the following tasks:

simple TCP proxies

shell-script based HTTP clients and servers

network daemon testing

a SOCKS or HTTP ProxyCommand for ssh(1)

The nc command can also be run as netcat, using the identical options.

Options The following options are supported:

- 4 Force nc to use IPv4 addresses only.
- 6 Force nc to use IPv6 addresses only.
- b bufsize Specify buffer size for read operations.

The default value is 1024 bytes.

- D Enable debugging on the socket.
- d Do not attempt to read from stdin.

-E Use exclusive bind for listening TCP or UDP socket.

It is an error to use this option without the -l option.

This option does not have any effect when used in conjunction with the -U option.

-e program Execute external program after accepting a connection or making connection. Before the execution stdin, stdout, stderr is redirected to the network descriptor. Only one port can be used with this option.

It is an error to use this option in conjunction with the -R, -k, or -i options.

-F Do not close network socket for writing after seeing EOF on stdin.

-h Print nc help.

–l bufsize Set receive (input) socket buffer size.

This option does not have any effect when used in conjunction with the –U option.

–i interval Specify a delay time of interval between lines of text sent and received.

The interval is specified in seconds, with possible fractions.

This option also causes a delay time between connections to multiple ports, and therefore also affects port scan mode.

–k Force nc to listen for another connection after its current connection is closed.

It is an error to use this option without the –l option.

It is an error to use this option in conjunction with the `-e` option.

`-L timeout` Linger on close - wait for messages to be sent after network descriptor is closed up to specified timeout in seconds.

`-l` Listen for an incoming connection rather than initiate a connection to a remote host.

It is an error to use this option in conjunction with the `-s` or `-z` options.

If the `-l` option is used with a wildcard socket (no IP address or hostname specified) and without the `-4` / `-6` options, it accepts both IPv4 and IPv6 connections.

`-m byte_count` Quit after receiving at least `byte_count` bytes. When used with `-l` option

byte_count is compared to number of bytes received from the client.

byte_count must be greater than 0 and less than INT_MAX.

-N file Specifies file with pattern for UDP port scanning. The contents of this file are used as payload for each emitted UDP packet.

It is an error to use this option without the -u and -z options.

-n Do not do any naming or service lookups on any addresses, hostnames, or ports.

Use of this option means that hostname and port arguments are restricted to numeric values.

If used with -v option all addresses and ports are printed in numeric form, in addition to

the restriction imposed on the arguments. This option does not have any effect when used in conjunction with the `-U` option.

`-O bufsize` Set send (output) socket buffer size.

This option does not have any effect when used in conjunction with the `-U` option.

`-P proxy_username` Specify a username (proxy_username) to present to a proxy server that requires authentication. If proxy_username is not specified, authentication is not attempted. Proxy authentication is only supported for HTTP CONNECT proxies at present.

It is an error to use this option in conjunction with the `-l` option.

`-p port` When used without `-l` option, specify the source port `nc` should use, subject to privilege restrictions and availability. When used with the `-l` option, set the listen port.

This option can be used with `-l` option only provided global port argument is not specified.

`-q timeout` After receiving EOF on stdin, wait for specified number of seconds and quit.

`-R addr/port[/proto]` Perform port redirection to given host and port.

After the connection has been accepted, `nc` connects to the remote host/port and passes all data between the client and the remote host. The `proto` (protocol) part of the redirect specification can be either `tcp` or `udp`. If the `proto` is not specified, redirector uses the same protocol as the server.

It is an error to use this option in conjunction with the `-z` option.

`-r` Choose destination ports randomly instead of sequentially within all ports specified by the `port_list` argument.

It is an error to use this option in conjunction with the `-l` option.

`-s source_ip_address` Specify the IP of the interface which is used to send the packets.

It is an error to use this option in conjunction with the `-l` option.

`-S sla-prop` Specify properties of the MAC flow created for the socket. `sla-prop` is supplied as a comma-separated list of 'name=value' of properties.

Currently supported property names are `maxbw`, `priority`, and `inherit`.

maxbw and priority come from the properties defined in flowadm(1M) and denote maximum bandwidth and priority of the flow. Allowed values for maxbw are integer plus optional suffix, which defaults to Mega. priority can take values from 'high', 'medium' and 'low'.

At least one of maxbw and priority need to be specified for flow creation.

inherit can take values from 'on' and 'off' with default value of 'off'. By default, an accepted/new socket (returned from accept(3C)) does not inherit the properties of the listener socket. When it is set to 'on', the new socket will inherit the properties of the listener socket. This is useful with -l option when the properties need to be enforced on the new socket.

This option requires SYS_FLOW_CONFIG privilege. This option also requires the IP address or the hostname to be specified.

-T dscp Specify Differentiated Services Code Point for the connection.

For IPv4 this specifies the IP Type of Service (ToS) IP header field and the valid values for the argument are the string tokens: lowdelay, throughput, reliability, or an 8-bit hexadecimal value preceded by 0x.

For IPv6 (Traffic Class) only hexadecimal value can be used.

-t Cause nc to send RFC 854 DON'T and WON'T responses to RFC 854 DO and WILL requests. This makes it possible to use nc to script telnet sessions.

`-U` Specify the use of Unix Domain Sockets. If you specify this option without `-l`, nc, it becomes AF_UNIX client. If you specify this option with the `-l` option, a AF_UNIX server is created.

Use of this option requires that a single argument of a valid Unix domain path has to be provided to nc, not a host name or port.

`-u` Use UDP instead of the default option of TCP.

`-v` Specify verbose output.

`-w timeout` Silently close the connection if a connection and stdin are idle for more than timeout seconds.

The default is no timeout.

This option has no effect on the connection establishment phase in client mode or

waiting for a connection in server mode.

`-X proxy_protocol` Use the specified protocol when talking to the proxy server. Supported protocols are 4 (SOCKS v.4), 5 (SOCKS v.5) and connect (HTTP proxy). If the protocol is not specified, SOCKS v. 5 is used.

It is an error to use this option in conjunction with the `-l` option.

`-x proxy_address[:port]` Request connection to hostname using a proxy at proxy_address and port. If port is not specified, the well-known port for the proxy protocol is used (1080 for SOCKS, 3128 for HTTP).

It is an error to use this option in conjunction with the `-l` option.

This option does not work with numeric representation of IPv6 addresses.

-Z In listening mode bind to address/port in all zones using the SO_ALLZONES socket option.

This option requires SYS_NET_CONFIG privilege.

-z Perform port scan. For TCP ports (default), connect scan (full 3-way handshake) is tried with no data sent. For UDP (-u) empty UDP packets are sent by default. To specify UDP payload the -N option can be used.

The UDP scan mode is estimative, it considers a port to be open if it does not receive negative response (ICMP Destination Port Unreachable message). For this mode the timeout set with the -w option is used to wait for the ICMP messages or data from remote node. With -v any received data is dumped as hexadecimal bytes to stderr.

As most of the operating systems employ rate limiting for sending ICMP messages in reaction to input packets, it is necessary to use `-i` when performing UDP scan otherwise the results is not reliable.

It is an error to use this option in conjunction with the `-l` option.

Operands The following operands are supported:

hostname Specify host name.

hostname can be a numerical IP address or a symbolic hostname (unless the `-n` option is specified).

In general, hostname must be specified, unless the `-l` option is given or `-U` is used (in which case the argument is a path). If hostname argument is specified with `-l`

option then port argument must be given as well and nc tries to bind to that address and port. If hostname argument is not specified with -l option then nc tries to listen on a wildcard socket for given port.

path Specify pathname.

port port_list Specify port.

port_list can be specified as single integers, ranges or combinations of both. Specify ranges in the form of nn-mm. The port_list must have at least one member, but can have multiple ports/ranges separated by commas.

In general, a destination port must be specified, unless the -U option is given, in which case a Unix Domain Socket path must be specified instead of hostname.

It is an error to use list of ports containing more than one port in conjunction with the -e option.

Usage Client/Server Model It is quite simple to build a very basic client/server model using nc. On one console, start nc listening on a specific port for a connection. For example, the command:

```
$ nc -l 1234
```

listens on port 1234 for a connection. On a second console (or a second machine), connect to the machine and port to which nc is listening:

```
$ nc 127.0.0.1 1234
```

There should now be a connection between the ports. Anything typed at the second console is concatenated to the first, and

vice-versa. After the connection has been set up, nc does not really care which side is being used as a server and which side is being used as a client. The connection can be terminated using an EOF (Ctrl/d).

Data Transfer The example in the previous section can be expanded to build a basic data transfer model. Any information input into one end of the connection is output to the other end, and input and output can be easily captured in order to emulate file transfer.

Start by using nc to listen on a specific port, with output captured into a file:

```
$ nc -l 1234 > filename.out
```

Using a second machine, connect to the listening nc process, feeding it the file which is to be transferred:

```
$ nc host.example.com 1234 < filename.in
```

After the file has been transferred, the connection closes automatically.

Talking to Servers It is sometimes useful to talk to servers by hand rather than through a user interface. It can aid in troubleshooting, when it might be necessary to verify what data a server is sending in response to commands issued by the client.

For example, to retrieve the home page of a web site:

```
$ echo -n "GET / HTTP/1.0\r\n\r\n" | nc  
host.example.com 80
```

This also displays the headers sent by the web server. They can be filtered, if necessary, by using a tool such as `sed(1)`.

More complicated examples can be built up when the user knows the format of requests

required by the server. As another example, an email can be submitted to an SMTP server using:

```
$ nc localhost 25 << EOF HELO
host.example.com MAIL FROM:
<user@host.example.com RCTP TO:
<user2@host.example.com DATA Body of
email. . QUIT EOF
```

Port Scanning It can be useful to know which ports are open and running services on a target machine. The `-z` flag can be used to tell nc to report open ports, rather than to initiate a connection.

In this example:

```
$ nc -z host.example.com 20-30 Connection
to host.example.com 22 port [tcp/ssh]
succeeded! Connection to host.example.com
25 port [tcp/smtp] succeeded!
```

The port range was specified to limit the search to ports 20 - 30.

Alternatively, it might be useful to know which server software is running, and which versions. This information is often contained within the greeting banners. In order to retrieve these, it is necessary to first make a connection, and then break the connection when the banner has been retrieved. This can be accomplished by specifying a small timeout with the `-w` flag, or perhaps by issuing a QUIT command to the server:

```
$ echo "QUIT" | nc host.example.com 20-30
SSH-2.0-Sun_SSH_1.1 Protocol mismatch.
220 host.example.com IMS SMTP Receiver
Version 0.84 Ready
```

inetd Capabilities One of the possible uses is to create simple services by using `inetd(1M)`.

The following example creates a redirect from TCP port 8080 to port 80 on host realwww:

```
cat << EOF >>  
/etc/services
```

```
wwwredir 8080/tcp # WWW redirect EOF
```

```
cat << EOF >  
/tmp/wwwredir.conf
```

```
wwwredir stream tcp nowait nobody  
/usr/bin/nc /usr/bin/nc -w 3 realwww 80 EOF
```

```
inetconv -i  
/tmp/wwwredir.conf
```

wwwredir ->

/var/svc/manifest/network/wwwredir-

tcp.xml Importing wwwredir-tcp.xml ...Done

inetadm -l wwwredir/tcp

SCOPE NAME=VALUE name="wwwredir"
endpoint_type="stream" proto="tcp"
isrpc=FALSE wait=FALSE exec="/usr/bin/nc -
w 3 realwww 80" arg0="/usr/bin/nc"
user="nobody" default bind_addr="" default
bind_fail_max=-1 default
bind_fail_interval=-1 default
max_con_rate=-1 default max_copies=-1
default con_rate_offline=-1 default
failrate_cnt=40 default failrate_interval=60
default inherit_env=TRUE default

tcp_trace=TRUE default

tcp_wrappers=FALSE

Privileges To bind to a privileged port number `nc` needs to be granted the `net_privaddr` privilege. If Solaris Trusted Extensions are configured and the port `nc` should listen on is configured as a multi-level port `nc` also needs the `net_bindmlp` privilege.

Privileges can be assigned to the user or role directly, by specifying them in the account's default privilege set in `user_attr(4)`.

However, this means that any application that this user or role starts have these additional privileges. To only grant the privileges(5) when `nc` is invoked, the recommended approach is to create and assign an `rbac(5)` rights profile. See **EXAMPLES** for additional information.

Examples Example 1 Using nc Open a TCP connection to port 42 of host.example.com, using port 3141 as the source port, with a timeout of 5 seconds:

```
$ nc -p 3141 -w 5 host.example.com 42
```

Open a TCP connection to port 7777 of host.example.com, setting a maximum bandwidth of 50Mbps on the socket:

```
$ nc -M maxbw=50M host.example.com  
7777
```

Open a UDP connection to port 53 of host.example.com:

```
$ nc -u host.example.com 53
```

Open a TCP connection to port 42 of host.example.com using 10.1.2.3 as the IP for the local end of the connection:

```
$ nc -s 10.1.2.3 host.example.com 42
```

Use a list of ports and port ranges for a port scan on various ports:

```
$ nc -z host.example.com 21-25,53,80,110-120,443
```

Create and listen on a Unix Domain Socket:

```
$ nc -lU /var/tmp/dsocket
```

Create and listen on a UDP socket with associated port 8888:

```
$ nc -u -l -p 8888
```

which is the same as:

```
$ nc -u -l 8888
```

Create and listen on a TCP socket with associated port 2222 and bind to address 127.0.0.1 only:

```
$ nc -l 127.0.0.1 2222
```

Create and listen on a TCP socket with associated port 2222 and create a high priority MAC flow on the listener and the connected sockets:

```
$ nc -l -M priority=high,inherit=on  
host.example.com 2222
```

Connect to TCP port, send some data and terminate the connection with TCP RST segment (instead of classic TCP closing handshake) by setting the linger option and timeout to 0:

```
$ echo "foo" | nc -L 0 host.example.com 22
```

Perform port redirection to port 22 on host host.example.com from local port 4545:

```
$ nc -R host.example.com/22 -l 4545
```

After that, it should be possible to run ssh(1) client

and connect to host.example.com using host
redir.example.com running the above
command:

```
$ ssh -oStrictHostKeyChecking=no -p 4545  
redir.example.com
```

It is also possible to let nc
listen on TCP port and convert the TCP data
stream to UDP (or vice versa):

```
$ nc -R host.example.com/53/udp -l 4666
```

Connect to port 42 of host.example.com
using an HTTP proxy at 10.2.3.4, port 8080.
This example could also be used by ssh(1) .
See the ProxyCommand directive in
ssh_config(4) for more information.

```
$ nc -x10.2.3.4:8080 -Xconnect  
host.example.com 42
```

The same example again, this time enabling
proxy authentication with username ruser if
the proxy requires it:

```
$ nc -x 10.2.3.4:8080 -Xconnect -Pruser  
host.example.com 42
```

Basic UDP port scan can be efficiently done like this:

```
$ nc -z -w 3 -u -i 0.5 host.example.com 11-100
```

Between each 2 ports it pauses for 0.5 second (thus evading ICMP message rate limiting) and waits up to 3 seconds for reply. If no reply comes then the port might be open.

To run nc with the smallest possible set of privileges as a user or role that has additional privileges (such as the default root account) it can be invoked using ppriv(1) as well. For example, limiting it to only run with the privilege to bind to a privileged port:

```
$ ppriv -e -  
sA=basic,!file_link_any,!proc_exec,!proc_fork  
,  
!proc_info,!proc_session,net_privaddr nc -l  
42
```

To allow a user or role to use only nc with the net_privaddr privilege, a rights profile needs to be created:

```
/etc/security/exec_attr Netcat  
privileged:solaris:cmd:::/usr/bin/nc:privs=net  
_privaddr
```

```
/etc/security/prof_attr Netcat  
privileged:::Allow nc to bind to privileged  
ports:help=None.html
```

Assigning this rights profile using user_attr(4) permits the user or role to run nc allowing it to listen on any port. To permit a user or role to use nc only to listen on

specific ports a wrapper script should be specified in the rights profiles:

```
/etc/security/exec_attr Netcat  
restricted:solaris:cmd:::/usr/bin/nc-  
restricted:privs=net_privaddr
```

```
/etc/security/prof_attr Netcat  
restricted:::Allow nc to bind to privileged  
ports:help=None.html
```

and write a shell script that restricts the permissible options, for example, one that permits one to bind only on ports between 42 and 64 (non-inclusive):

```
/usr/bin/nc-restricted:  
  
#!/bin/sh [ $# -eq 1 ] && [ $1 -gt 42 -a $1 -lt  
64 ] && /usr/bin/nc -l -p "$1"
```

This grants the extra privileges when the user or role invokes nc using the wrapper

script from a profile shell. See `pfsh(1)`, `pfksh(1)`, `pfcsh(1)`, and `pfexec(1)`.

Invoking `nc` directly does not run it with the additional privileges, and neither does invoking the script without using `pfexec` or a profile shell.

Attributes See `attributes(5)` for descriptions of the following attributes:

ATTRIBUTE TYPE ATTRIBUTE VALUE

Availability network/netcat Interface

Stability See below. The package name is

Committed. The command line syntax is

Committed for the `-4`, `-6`, `-l`, `-n`, `-p`, `-u`, and `-w` options and their arguments (if any). The

name and port list arguments are

Committed. The port range syntax is

Uncommitted. The interface stability level

for all other command line options and their arguments is Uncommitted.

See Also cat(1), pfcsh(1), pfexec(1), pfksh(1), pfsh(1) , ppriv(1), sed(1), ssh(1), telnet(1), inetadm(1M), inetconv(1M), inetd(1M), ssh_config(4), user_attr(4), attributes(5), privileges(5), rbac(5)

Authors The original implementation of nc was written by Hobbit, hobbit@avian.org.

nc was rewritten with IPv6 support by Eric Jackson, ericj@monkey.org.

Notes If an instance of nc is listening on a wildcard socket (regardless of address family specification) it is still possible to bind another nc process to concrete IP address and accept connections to this address. For example, with the following process running:

\$ nc -4 -l 5656 it is possible to run another nc process listening on specific IP address and the same port:

\$ nc -4 -l 10.20.30.40 5656 TCP connection to address 10.20.30.40 and port 5656 is accepted by the latter process, all TCP connections to port 5656 and different addresses is accepted by the former process.

Also, it is possible to steal IPv4 connections from a process which listens on a wildcard socket (without address family specification) by binding to IPv4 wildcard socket. To suppress this and the behavior described above the `-E` option could be used.

Copyright © 1993, 2017, Oracle and/or its affiliates. All rights reserved.